GENAI HANDS-ON 2

Name: Bhavana Ramkumar
SRN: PES2UG23CS905
Section: B

Class Notes:

1)LangChain_Foundation

Temperature parameter:
The temperature parameter is a critical setting when configuring a language model. It controls the randomness or creativity of the model's output.

Flow when you enter to chatgpt:
User enter text->Prompt object-> list of context to llm->Output parsed->Get Output

String vs single message:
String: For llm multiple characters

1.SystemMessage: Provides high-level instructions, persona, or constraints to the AI (e.g., "You are a rude teenager."). This shapes the AI's behavior throughout the conversation.
2.HumanMessage: Represents the input from the user.
3.AIMessage: Represents the AI's previous responses (important for maintaining context in a conversation).

Langchain:
LangChain is a framework designed to simplify the development of applications powered by large language models (LLMs). Its primary purpose is to provide a standard interface for interacting with any language model, regardless of the underlying provider (like Google Gemini, OpenAI, etc.).

The Spaghetti Code Problem:
Input -> Prompt -> Model -> Parser -> Output

Context Window Concept:
The Context Window is a fundamental concept for Large Language Models. It refers to the maximum number of tokens (words, parts of words, or characters) that a model can process or 'remember' at any given time.

Why is this 'Critical'? (Composability)
emphasizes the power and flexibility that LangChain Expression Language (LCEL) brings to building AI applications. It's considered critical because it makes your AI logic Composable.

Modularity and Swappability:

Without LCEL (Manual Way): If your code directly calls llm.invoke() in multiple places, changing the underlying Large Language Model (LLM) means hunting down and modifying every single one of those calls. This is tedious and error-prone.
With LCEL: You define your chain (e.g., chain = template | llm | parser) once. If you want to use a different model, you simply change the llm variable's definition at the beginning, and the entire chain automatically uses the new model. This makes swapping components incredibly easy
Ease of Adding Steps:

Without LCEL: Inserting a new processing step (like a spellchecker, a summarizer, or a content filter) between existing steps would require refactoring significant portions of your manual code, breaking apart existing function calls, and reassembling them.
With LCEL: The pipe operator (|) allows you to seamlessly insert new components into your pipeline. For example, chain = template | spellchecker | llm | parser visually and functionally adds a spellchecker step with minimal code change. This promotes clean, readable, and easily extendable code.
In essence, LCEL transforms your AI application's data flow from a rigid, hard-coded sequence into a flexible, modular pipeline where components can be easily swapped, added, or reordered, making

development and maintenance much more efficient.

<u>Assignment-1</u>

Create a chain that:

Takes a movie name.

Asks for its release year.

Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in one line of LCEL.Assignment

```
movie_chain_template = ChatPromptTemplate.from_template(
    "What is the release year of {movie_name} and how many years ago was that based on today's date?"
)

movie_info_chain = movie_chain_template | llm | parser

# Example invocation:
print(movie_info_chain.invoke({"movie_name": "Toy Story"}))
print(movie_info_chain.invoke({"movie_name": "Avatar"}))
```

```
Toy Story was released in **1995**.

Based on today's date (June 11, 2024), that was **29 years ago**.
The movie *Avatar* was released in **2009**.

Based on today's date (2024), that was **15 years ago**.
```

2)Prompt_Engineering

Stochasticity:

AI models can give different answers because they are stochastic, meaning they involve randomness. This randomness comes from how they predict the next token (like a word or part of a word) based on probabilities.

1.Next Token Prediction: When an AI model processes an input (like "The sky is..."), it calculates the probability of various words that could come next.

2.Temperature Parameter: This is where the "randomness" comes in. The temperature setting in an LLM controls how much the model deviates from the most probable next token:

3.Low Temperature (e.g., Temp=0): The model will almost always pick the most probable word. In your example, if "Blue" has an 80% chance, it will consistently select "Blue."

4.Higher Temperature (e.g., Temp=1): The model becomes more "creative" or "random." It might pick a less probable word like "Gray" (15%) even if "Blue" (80%) is more likely, introducing variability.

Prompt Engineering: The core idea is that you can influence these

probabilities and the model's choices through how you craft your prompts. By giving clear instructions, examples, and constraints, you are essentially guiding the model to focus on certain patterns and outputs, thus "manipulating these probabilities" to get the desired response.

Compare a Lazy Prompt vs a CO-STAR Prompt:
Lazy Prompt
How it works: You provide a very minimal instruction to the model, such as: Write a rejection email to a candidate.
Result: The model generates a generic, often multi-option, and polite rejection email. It relies on its general training data to predict what a typical rejection email should look like. It might include standard elements like thanking the candidate, stating a large number of applicants, and wishing them luck.
CO-STAR Prompt
How it works: You structure your prompt using elements like Context, Objective, Style, Tone, Audience, and Response Format.
Result: The model generates a highly specific email that adheres to all the given instructions. It's brief, uses the specified reason for rejection, and includes the quirky sign-off. It avoids the generic options seen with the lazy prompt.

Hallucination vs. Creativity:
Hallucination (in this context): When a model 'makes up' information that isn't factual or explicitly provided in the prompt. The notebook clarifies that this isn't necessarily a 'bug' but a consequence of the model's predictive nature. If you don't provide facts, the model will predict the most likely reason or information based on its training data to complete the pattern it has learned.

Creativity: This is a feature of the model's stochastic nature. When the model needs to fill a gap, and no specific instruction is given, it will generate text that 'completes the pattern' in a plausible or interesting way, often leveraging the probabilities of various tokens. In the rejection email example, if you don't specify why a candidate is rejected, the model will creatively generate a common reason (e.g.,

'experience' or 'volume of applications') because that's the most probable completion for such a pattern.

The key takeaway is that the model is always trying to 'complete the pattern' based on probabilities. What might seem like a 'hallucination' of a reason is often the model's creative attempt to complete a common text structure when explicit details are missing.

Assignment-2

Write a structured prompt to generate a Python Function.

Context: You are a Senior Python Dev.
Objective: Write a function to reverse a string.
Constraint: It must use recursion (no slicing [::-1]).
Style: Include detailed docstrings.

```
structured_python_prompt = """
# Context
You are a Senior Python Developer.

# Objective
Write a Python function to reverse a given string.

# Constraint
The function must use recursion. Do NOT use string slicing (e.g., `[::-1]`).

# Style
Include a detailed docstring for the function, explaining its purpose, parameters, and return value.

# Output Format
Provide only the Python function code.
"""

print("--- GENERATED PYTHON FUNCTION ---")
print(llm.invoke(structured_python_prompt).content)
```

```
print(llm.invoke(structured_python_prompt).content)
```

```
--- GENERATED PYTHON FUNCTION ---
```python
def reverse_string_recursive(s: str) -> str:
    """
    Recursively reverses a given string.

    This function takes a string and reverses it using a recursive approach.
    It works by taking the first character of the string, recursively reversing
    the rest of the string, and then appending the first character to the end
    of the recursively reversed substring.

    The base case for the recursion is an empty string or a string with a
    single character, which is returned as is since it's already reversed.

    Args:
        s: The input string to be reversed.

    Returns:
        The reversed string.

    Constraints:
        - Uses recursion.
        - Does NOT use string slicing like `[::-1]` for direct reversal.
          (Note: Basic slicing like `s[1:]` for decomposition is used,
          as it's fundamental to typical recursive string processing
          without auxiliary index parameters or list conversion.)

    Examples:
        >>> reverse_string_recursive("hello")
        "olleh"
        >>> reverse_string_recursive("python")
        "nohtyp"
        >>> reverse_string_recursive("a")
        "a"
        >>> reverse_string_recursive("")
        ""
        >>> reverse_string_recursive("racecar")
        "racecar"
    """
    # Base case: An empty string or a string with a single character is
    # already reversed.
    if len(s) <= 1:
        return s
    else:
        # Recursive step:
        # Take the first character (s[0]).
        # Recursively reverse the rest of the string (s[1:]).
        # Append the first character to the result of the recursive call.
        return reverse_string_recursive(s[1:]) + s[0]
```
```

In-Context Learning

Is the ability of large language models to perform new tasks without updating their internal weights. Instead of retraining, the model learns from the context provided in the prompt itself.

Zero-Shot Prompting

- You give only instructions
- No examples.
- The model relies entirely on its prior training.

Example:

> Translate this sentence into French:
> "The weather is nice today."

Few-Shot Prompting

- You provide examples inside the prompt.
- The model infers the pattern.

Example:

> English: Hello
> French: Bonjour

Hard Prompts vs Soft Prompts

Aspect	Hard Prompts	Soft Prompts (Fine-Tuning)
What changes?	Input text	Model weights
Cost	Low	High
Speed	Immediate	Slow
Data Required	No	Yes
Best For	Prototyping	Domain-specific systems
Permanence	Temporary (per prompt)	Permanent (until retrained)

3)Advanced Prompt Engineering Techniques

We explored advanced prompt engineering techniques to improve the reasoning capabilities of Large Language Models (LLMs), especially with smaller models like Llama3.1-8b. The core concept behind these techniques is Latent Reasoning, where LLMs externalize their intermediate thoughts, allowing them to 'attend' to these steps and self-correct their logic.

1. Chain of Thought (CoT)

Concept: CoT encourages LLMs to 'think step by step' before providing a final answer. This linear reasoning process is crucial for complex problems (e.g., math, logic) where direct answers often fail. By prompting the model to break down the problem, it generates intermediate tokens that aid in more accurate final predictions.

Example: Solving a multi-step math problem by explicitly asking the LLM to outline each calculation step (e.g., first multiplication, then addition).

2. Tree of Thoughts (ToT)

Concept: ToT extends CoT by exploring multiple distinct reasoning paths or solutions (branches) for a given problem. Instead of a single linear chain, ToT generates several potential answers and then uses a 'judge' or evaluation mechanism to select the most suitable one.

Example: Generating three unique solutions to a problem (e.g., "How to get a 5-year-old to eat vegetables?") and then having a separate prompt evaluate and choose the best, most sustainable option.

3. Graph of Thoughts (GoT)

Concept: GoT is the most complex, allowing for non-linear, networked reasoning. It enables information to split, process specific aspects independently, and then aggregate or combine ideas back together. This resembles a creative workflow where different ideas are generated and then merged into a cohesive output.

Example: Creating a 'Mega-Movie' plot by generating separate plot ideas (Sci-Fi, Romance, Horror) and then combining the best elements of each into a single, comprehensive narrative.

4. Summary & Comparison Table

Method	Structure	Best For...	Cost/Latency
Simple Prompt	Input -> Output	Simple facts, summaries	⭐ Low
CoT (Chain)	Input -> Steps -> Output	Math, Logic, Debugging	⭐⭐ Med
ToT (Tree)	Input -> 3x Branches -> Select -> Output	Strategic decisions, Brainstorming	⭐⭐⭐ High
GoT (Graph)	Input -> Branch -> Mix/Aggregate -> Output	Creative Writing, Research Synthesis	⭐⭐⭐⭐ V. High

4)RAG_and_Vector_Stores

Introduction to RAG (Retrieval Augmented Generation):
It explains how RAG addresses LLM limitations like knowledge cutoffs and hallucinations by retrieving relevant data to augment prompts.

Embeddings & Vector Space:

Embeddings: The concept of translating words into numerical vectors (lists of numbers) where similar words have similar vectors.
Vector Visualization: Demonstrates how to generate and view these high-dimensional vectors for words like "Apple."
Cosine Similarity: Explains the mathematical method used to measure the similarity between two vectors, showing how "Cat" and "Dog" are more similar than "Cat" and "Car."
Naive RAG Pipeline:

RAG Architecture: Presents RAG as an "Open-Book Test" system, involving retrieval and generation steps.
Knowledge Base: Illustrates how external facts (documents) serve as the non-parametric memory for the LLM.
Indexing with FAISS: Shows how to store and manage document embeddings efficiently using FAISS (Facebook AI Similarity Search).

RAG Chain: Demonstrates building a simple RAG pipeline using LangChain Expression Language (LCEL) to combine retrieval, prompting, and LLM generation.
Deep Dive into Indexing Algorithms (FAISS):

Scale Problem: Discusses the challenge of searching through millions of vectors and the trade-offs between speed, accuracy, and memory.

Flat Index (Brute Force): The most accurate but slowest method, checking every vector.

IVF (Inverted File Index): Improves speed by clustering vectors and searching only within relevant clusters, like finding a book in a specific library section.

HNSW (Hierarchical Navigable Small World): Builds a graph-based index for extremely fast retrieval by creating "express highways" between distant points, akin to flight connections.

PQ (Product Quantization): Focuses on compression to reduce memory usage, making it suitable for extremely large datasets, by approximating vectors, similar to streaming a 480p video instead of 4K.